



МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)

**Институт
информационных технологий**

**Кафедра
информационных систем**

Основная образовательная программа 09.03.02
«Информационные системы и технологии»

КУРСОВАЯ РАБОТА

по дисциплине «Объектно-ориентированное программирование»

Тема: «Магазин техники»

Руководитель
к.т.н., доцент

Разумовский А.И.

подпись

Выполнил
студент группы ИДБ-22-07

Румянцев М.С.

подпись

Москва, 2024

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
ГЛАВА 1. СРЕДА И СРЕДСТВА РАЗРАБОТКИ.....	4
1.1. СРЕДА РАЗРАБОТКИ	4
1.2. ЯЗЫК ПРОГРАММИРОВАНИЯ	7
1.3. ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ	10
1.4. MICROSOFT FOUNDATION CLASSES (MFC)	14
ГЛАВА 2. РАЗРАБОТКА ПРОГРАММНОГО ПРОДУКТА.....	17
2.1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ.....	17
2.2. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ	18
ЗАКЛЮЧЕНИЕ	29
СПИСОК ЛИТЕРАТУРЫ.....	30
ПРИЛОЖЕНИЕ 1.	31
ПРИЛОЖЕНИЕ 2.	36
ПРИЛОЖЕНИЕ 3.	38

ВВЕДЕНИЕ

Целью данной курсовой работы является реализация системы управления базой данных магазина техники. Для реализации этой цели требуется создать программу, способную фиксировать поступление товара на склад, сохраняя название, его стоимость, количество и производителя (если таковой указан).

База данных - это организованный набор данных, хранящийся в компьютерной системе и управляемый системой управления базами данных (СУБД). Она позволяет эффективно хранить, организовывать и искать информацию, обеспечивая централизованный доступ и контроль качества данных. Создание баз данных важно для организаций, так как повышает целостность данных, обеспечивает эффективное управление и помогает принимать обоснованные решения. Базы данных сокращают время поиска информации, автоматизируют задачи, связанные с управлением данными, и позволяют проводить анализ данных для принятия прогнозных решений. Они также помогают организациям соответствовать нормативным требованиям, обеспечивают безопасность и конфиденциальность данных, а также позволяют предоставлять персонализированное обслуживание клиентов.

Для разработки программного продукта было решено использовать язык программирования C++. Этот язык обладает высокой производительностью и гибкостью, что важно для реализации сложных проектов. C++ также поддерживает объектно-ориентированный подход к программированию, позволяющий создавать модульные и масштабируемые приложения.

ГЛАВА 1. СРЕДА И СРЕДСТВА РАЗРАБОТКИ

1.1. СРЕДА РАЗРАБОТКИ

Visual Studio (VS) - это интегрированная среда разработки (IDE), разработанная компанией Microsoft. С момента своего появления в 1997 году, VS стала одной из наиболее популярных и широко используемых сред разработки в мире [4]. Ее популярность обусловлена широким спектром функциональных возможностей, интуитивно понятным интерфейсом и высокой производительностью. Visual Studio предоставляет разработчикам набор инструментов и функций для создания, тестирования и развертывания различных типов приложений, включая десктопные, web-приложения, мобильные приложения и игры.

Visual Studio предоставляет разработчикам набор инструментов и функций для создания, тестирования и развертывания различных типов приложений, включая десктопные, web-приложения, мобильные приложения и игры. Среди основных функций Visual Studio можно выделить редактор кода с поддержкой различных языков программирования, интеллектуальным автодополнением кода, подсветкой синтаксиса и рефакторингом, дизайнер интерфейсов для визуального проектирования пользовательского интерфейса приложений, встроенную систему управления версиями (поддержка Git и других популярных систем) для отслеживания изменений в коде, восстановления предыдущих версий и совместной работы над проектами, средства тестирования для создания и запуска юнит-тестов, интеграционных тестов и других видов тестирования, а также возможности деплоя и развертывания для публикации приложения в различных средах (локально, в облаке, на сервере) с интегрированной поддержкой различных платформ для хостинга и деплоя. Благодаря комплексному подходу, Visual Studio является незаменимым инструментом для разработчиков, позволяющим выполнять весь процесс разработки в одной среде, повышая эффективность и продуктивность.

Таким образом, Visual Studio предлагает разработчикам все необходимые инструменты для эффективной разработки, тестирования и развертывания программного обеспечения.

Редактор кода.

Редактор кода в Visual Studio — это мощный инструмент для разработки программного обеспечения, доступный на платформах Windows, macOS и Linux. Он обладает множеством полезных функций, включая подсветку синтаксиса для множества языков программирования, автоматическое завершение кода, интегрированный отладчик и поддержку системы контроля версий Git. Visual Studio Code также предлагает обширную библиотеку расширений, которые позволяют пользователям легко настроить редактор под свои потребности и предпочтения. Эти расширения предоставляют поддержку различных языков программирования, фреймворков, инструментов и библиотек, что делает разработку более эффективной и удобной. Кроме того, пользователи могут создавать собственные расширения, используя открытый исходный код Visual Studio Code. Это позволяет разработчикам добавлять новые функции и возможности, которые не доступны в стандартных расширениях. Встроенный терминал в Visual Studio Code позволяет выполнять команды и скрипты прямо из редактора, что упрощает процесс разработки и сокращает время на переключение между инструментами. Редактор кода в Visual Studio работает быстро и стабильно даже на компьютерах с невысокой мощностью, что делает его доступным для начинающих разработчиков.

Дизайнер интерфейсов.

Дизайнер интерфейсов в Visual Studio — это инструмент, позволяющий создавать макеты пользовательского интерфейса для различных платформ, таких как Windows, macOS и Android. Он интегрирован с Visual Studio и предоставляет возможность визуального проектирования интерфейсов с использованием библиотеки материалов Microsoft. Основные функции дизайнера интерфейсов включают создание и редактирование элементов интерфейса, настройку свойств элементов, импорт и экспорт файлов, а также

предварительный просмотр интерфейса на разных устройствах и разрешениях экрана. Дизайнер поддерживает различные языки программирования и технологии, такие как HTML, CSS, JavaScript, Angular и React. Для удобства работы дизайнер интерфейсов предлагает инструменты для быстрого доступа к часто используемым элементам и функциям, а также возможность настройки горячих клавиш для быстрого выполнения операций.

Система управления версиями.

Система управления версиями в Visual Studio позволяет разработчикам легко управлять своим кодом и работать вместе. Возможно использование Git для распределённого управления версиями или Team Foundation Version Control (TFVC) для централизованного управления версиями прямо из коробки.

Visual Studio также поддерживает интеграцию с популярными системами управления версиями, такими как GitHub и Azure DevOps, и позволяет легко добавлять свой код в эти сервисы.

Средства тестирования.

Инструменты тестирования в Visual Studio предлагают разработчикам широкий спектр возможностей для проверки качества кода и обнаружения ошибок. Среди них модульное тестирование, IntelliTest, Live Unit Testing, обозреватель тестов, индикаторы тестов CodeLens и анализ объёма протестированного кода. Эти инструменты позволяют разработчикам создавать высококачественный код, обнаруживать и исправлять ошибки, обеспечивая высокое качество продукта и удовлетворяя потребности пользователей.

Преимущества Visual Studio.

Visual Studio имеет ряд преимуществ: комплексный подход, высокую производительность, интуитивный интерфейс, большую экосистему, поддержку Microsoft, разные языки программирования, интеграцию с системами контроля версий, разработку с использованием искусственного

интеллекта, создание кроссплатформенных приложений и адаптивных веб-интерфейсов.

1.2. ЯЗЫК ПРОГРАММИРОВАНИЯ

Язык программирования - это своеобразный инструмент, который позволяет программистам создавать инструкции для компьютера, чтобы он мог их понять и выполнить. Этот язык представляет собой не просто набор символов и правил, а целую систему, с помощью которой разработчики могут формулировать различные алгоритмы и логику для решения разнообразных задач.

Программа, в свою очередь, представляет собой набор таких инструкций, написанных на выбранном языке программирования, которые компьютер последовательно выполняет, чтобы достичь определенного желаемого результата. Это как набор шагов, которые компьютер должен выполнить в определенной последовательности, чтобы сделать что-то конкретное.

Один из таких языков программирования - C++. Этот высокоуровневый язык общего назначения был разработан известным ученым Бьярном Страуструпом в начале 1980-х годов. C++ является компилируемым языком, что означает, что код, написанный на нем, сначала преобразуется в машинный код, который уже может быть выполнен компьютером [2]. Таким образом, программисты могут использовать C++ для создания эффективных и мощных программ, которые выполняют различные задачи с высокой производительностью и точностью.

Язык программирования C++ — это один из самых востребованных инструментов в сфере создания программного обеспечения. Его применение охватывает разнообразные области, включая разработку операционных систем, прикладного софта, игр, мобильных приложений, алгоритмов и многое другое. C++ широко применяется в различных сферах, таких как операционные системы, технологические компании, финансовые учреждения,

игровая индустрия и другие отрасли благодаря своей выдающейся производительности, гибкости и масштабируемости.

Этот язык является компилируемым, что означает, что программы на C++ преобразуются в машинный код, что обеспечивает высокую скорость работы и эффективное использование ресурсов компьютера. Компиляторы C++ поддерживаются на различных операционных системах, что позволяет программам, написанным на C++, легко работать на различных платформах.

В C++ разработчики имеют широкий набор возможностей, таких как работа с различными структурами данных, управление памятью, объектно-ориентированное программирование, шаблоны, обработка исключений, многопоточное программирование и многое другое. Все это делает C++ одним из самых гибких и мощных языков программирования, предоставляя разработчикам широкие возможности для воплощения своих идей в жизнь.

Для эффективного хранения информации в программировании на языке C++ необходимо использовать переменные. По своей сути переменная представляет собой именованный участок памяти, который обладает типом, наименованием и значением. Выбранный тип переменной определяет, какие данные могут быть в ней сохранены, что обеспечивает строгую типизацию в языке программирования C++.

Основываясь на принципах объектно-ориентированного программирования (ООП), C++ позволяет разработчикам создавать высоко модульные и масштабируемые программные продукты. Возможности ООП в C++ реализуются через использование классов, объектов, механизмов наследования, инкапсуляции и полиморфизма, что делает данный язык программирования мощным инструментом для построения сложных информационных систем.

C++ широко применяется в разработке игр благодаря своей высокой производительности, а также наличию полезных библиотек и инструментов, таких как Unreal Engine и Unity, которые предоставляют удобные средства для создания игровых приложений на данном языке. Множество известных игр и

игровых движков были разработаны с использованием C++ или же используют его в качестве основы.

Одним из важных аспектов C++ является возможность перегрузки операторов, что дает разработчикам возможность определить специальное поведение для стандартных операторов в зависимости от типов данных [2]. Это придает гибкости и удобства при работе с пользовательскими классами и различными типами информации.

C++ - язык программирования, который обладает рядом важных особенностей, среди которых стоит выделить возможность работы на низком уровне с памятью и указателями [2]. Благодаря этой особенности разработчики имеют возможность более детально управлять ресурсами и оптимизировать производительность своих программ. Однако стоит помнить, что для этого требуется более внимательное и аккуратное программирование, чтобы избежать ошибок.

Кроме того, C++ широко применяется на практике для создания высокопроизводительных приложений и систем в различных областях. Например, его используют для разработки финансовых платформ, игровых движков, медицинского программного обеспечения, алгоритмов и многих других проектов. Этот язык остается одним из основных инструментов программирования, который выбирают для создания критически важных и сложных систем.

Все перечисленные особенности делают C++ мощным и универсальным языком программирования, способным удовлетворить потребности разработчиков в самых разнообразных областях. Создание и поддержка проектов на C++ являются ключевыми аспектами инфраструктуры многих компаний и структур, что подчеркивает его важность и актуальность в современном мире разработки программного обеспечения.

1.3. ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ

Объектно-ориентированное программирование (ООП) является широко используемым методом разработки программного обеспечения. ООП предлагает ряд преимуществ, включая повышенную модульность, повторное использование кода и упрощение сопровождения программ. Истоки ООП уходят в прошлое, вплоть до 1960-х и 1970-х годов. В этот период Симула был разработан в Норвежском центре для вычислительной математики в Осло [3].

Следует отметить, что объектно-ориентированное программирование основано на нескольких ключевых принципах, таких как наследование, полиморфизм и инкапсуляция. Наследование позволяет создавать новые классы, наследуя данные и методы от существующих классов. Полиморфизм позволяет объектам работать с различными типами данных одинаковым образом. И инкапсуляция обеспечивает сокрытие деталей реализации, что приводит к лучшей структуре кода.

Разработка программного продукта на основе ООП требует умения абстрагировать проблему, выделять сущности и определять связи между ними. Это позволяет описать предметную область в терминах объектов, что существенно упрощает процесс моделирования.

Кроме того, парадигма ООП привела к широкому распространению понятий классов, наследования, полиморфизма, инкапсуляции и абстракции. Важно отметить, что ООП стал интегральной частью многих современных языков программирования, включая C++, что делает его ключевым аспектом для разработчиков ПО.

Классы.

Классы играют фундаментальную роль в объектно-ориентированном программировании (ООП) и являются ключевым аспектом создания высококачественного и эффективного программного обеспечения [5]. В рамках ООП, класс представляет собой шаблон или форму для создания объектов, объединяющих данные и методы, которые оперируют этими данными. Через классы реализуется принцип абстракции, непосредственно

связанный с выделением ключевых характеристик и поведения объектов, что делает его фундаментальным понятием в ООП.

Каждый класс определяет состояние (поля) и поведение (методы) объектов. Поля класса представляют собой переменные, используемые для хранения данных, а методы - функции, которые действуют на эти данные.

Создание экземпляров классов (объектов) позволяет использовать данные и методы, определенные в классе. Объекты - это конкретные экземпляры класса, которые могут содержать свои собственные значения для полей класса.

Конструктор класса играет важную роль в создании объектов. Он выполняет инициализацию объекта при его создании, устанавливая начальные значения для его полей. Различные языки программирования предоставляют различные способы определения и использования конструкторов, но их основная цель одинакова - обеспечить корректное состояние нового объекта.

Таким образом, классы в рамках ООП обеспечивают структурирование данных и поведения в программном коде, позволяя создавать объекты с определенными характеристиками и функциональностью.

Наследование.

Наследование - это важное понятие в объектно-ориентированном программировании (ООП), которое позволяет создавать новые классы на основе уже существующих [3]. В контексте ООП, существует иерархия классов, где классы-потомки получают свойства и методы от своих родительских классов, обеспечивая повторное использование кода и уменьшая дублирование. Наследование способствует повышению уровня абстракции, структурированию кода и облегчает обслуживание программ.

Родительский класс или суперкласс - это класс, от которого наследуют другие классы, называемые дочерними или производными классами. В процессе наследования, дочерний класс расширяет функциональность родительского класса, добавляя свои собственные методы и поля. Наследование способствует созданию иерархии классов, где классы более

общего типа могут выступать в качестве родителей для более специализированных классов.

Таким образом, наследование позволяет создавать и поддерживать иерархии классов, что обеспечивает повторное использование кода и упрощает создание и обслуживание программного обеспечения. Оно является важной концепцией в объектно-ориентированных системах, способствуя модульности, повторному использованию и расширяемости программного кода.

Полиморфизм.

Полиморфизм - важная концепция в объектно-ориентированном программировании (ООП), которая обеспечивает гибкость и расширяемость кода, упрощает его поддержку и содействует созданию более абстрактных и эффективных систем [5]. Полиморфизм позволяет объектам различных классов обладать одинаковым интерфейсом, т.е. использовать функции с одним и тем же именем, но с разной реализацией, в зависимости от типа объекта.

Один из принципов полиморфизма - это переопределение методов родительских классов в дочерних классах. Это означает, что дочерний класс может предоставить собственную реализацию метода, который унаследован от родительского класса. Таким образом, при вызове одного и того же метода для объектов разных классов, будет вызываться соответствующая реализация этого метода в каждом из классов.

Одной из форм реализации полиморфизма является идея интерфейсов. Интерфейс определяет набор методов, которые должны быть реализованы классами, которые реализуют этот интерфейс. Это позволяет утверждать, что все классы, реализующие интерфейс, обладают одним и тем же набором методов, но реализацию этих методов они могут осуществлять каждый по-своему.

Таким образом, полиморфизм в ООП обеспечивает возможность обработки объектов различных типов через общий интерфейс, улучшая

читаемость, гибкость и повторное использование кода. Он играет ключевую роль в разработке гибких и адаптивных программных систем, позволяя программистам создавать более абстрактные и расширяемые решения.

Инкапсуляция.

Инкапсуляция является одним из основных принципов объектно-ориентированного программирования (ООП) и представляет собой механизм объединения данных и методов в единое целое [3]. Это позволяет скрыть детали реализации объекта от внешнего мира, обеспечивает защиту данных и управление доступом к ним, что повышает безопасность и уменьшает вероятность ошибок в программном коде.

В рамках инкапсуляции переменные класса обычно делаются приватными, что означает, что они доступны только внутри самого класса. Доступ к этим переменным осуществляется через методы класса, так называемые геттеры и сеттеры. Геттеры используются для получения значения переменных, а сеттеры для установки новых значений. Такой подход позволяет контролировать чтение и запись данных, обеспечивая их целостность и безопасность.

Инкапсуляция способствует упрощению использования классов в программировании, поскольку она позволяет создавать четкие интерфейсы для работы с объектами. Это значительно уменьшает объем информации, с которой нужно взаимодействовать при использовании класса, так как детали его реализации скрыты за пределами класса. Таким образом, программисты могут сосредоточиться на использовании объектов, не заботясь о том, как эти объекты устроены внутри.

Инкапсуляция также содействует повышению безопасности программного кода, поскольку ограничивает доступ к данным и методам класса, предотвращая случайное вмешательство и неправомерное изменение значений. Благодаря инкапсуляции код становится более надежным, устойчивым к ошибкам и легче поддерживаемым в долгосрочной перспективе.

Все это делает инкапсуляцию обязательным элементом при проектировании качественного и эффективного программного обеспечения в рамках ООП.

Абстракция.

Абстракция в рамках объектно-ориентированного программирования (ООП) представляет собой процесс выделения общих характеристик объекта, игнорируя его детали реализации, и приведение этих характеристик к форме моделей, которые можно использовать для создания абстрактных классов и интерфейсов [3]. Абстракция позволяет создавать обобщенные модели объектов, которые могут быть использованы для определения общих поведенческих и структурных характеристик, и, таким образом, повышает уровень абстракции при разработке программного обеспечения.

Основная идея абстракции состоит в том, чтобы сконцентрироваться на важных свойствах объектов, игнорируя их конкретные реализации.

Абстракция обычно реализуется через абстрактные классы и интерфейсы в ООП. Абстрактные классы содержат набор характеристик и методов, которые могут быть унаследованы другими классами, тогда как интерфейсы представляют собой чистые абстрактные модели, содержащие только сигнатуры методов, но не их реализацию.

Использование абстракции позволяет разрабатывать более обобщенные, гибкие и масштабируемые приложения. Оно также способствует иерархичному проектированию программ, позволяя создавать модели высокого уровня, которые могут быть адаптированы для различных потребностей и условий, обеспечивая повторное использование кода и уменьшая сложность программного обеспечения.

1.4. MICROSOFT FOUNDATION CLASSES (MFC)

Microsoft Foundation Classes, более известные как MFC, являются мощной и функциональной библиотекой, написанной на популярном языке программирования C++, которая была разработана ведущей технологической компанией Microsoft с конкретной целью упрощения процесса разработки

графических пользовательских интерфейсов (GUI) для программ, которые были исключительно направлены на работу в рамках операционной системы Microsoft Windows [4]. Эта библиотека была создана с учетом требований разработчиков и была предназначена для того, чтобы значительно облегчить и ускорить процесс разработки приложений, предоставляя разработчикам обширный набор классов библиотеки, которые не только упрощают процесс создания привлекательных и функциональных пользовательских интерфейсов, но и позволяют легко управлять событиями, эффективно работать с файлами и базами данных, а также осуществлять множество других важных функций, связанных с разработкой приложений, которые были предназначены для использования в окружении операционной системы Windows.

Методы и классы MFC предлагают разработчикам мощную и гибкую объектно-ориентированную оболочку на языке C++, которая была специально сконструирована для операционной системы Windows. Это позволяет программистам эффективно и в кратчайшие сроки разрабатывать и реализовывать классические приложения, которые функционируют на языке машинного кода. Множество разработчиков выбирают библиотеку MFC для создания приложений, предназначенных для рабочего стола под управлением Windows, поскольку она обеспечивает высокую скорость и удобство в разработке. Множество разработчиков активно используют MFC для реализации разнообразных типов приложений, включая такие популярные формы, как единый интерфейс документа (SDI), несколько интерфейсов документов (MDI), приложения, основанные на диалоговых окнах, приложения, основанные на формах, а также библиотеки динамических связываемых модулей (DLL), что делает их еще более универсальными и пригодными для широкого спектра задач в сфере разработки программного обеспечения.

Использование архитектуры документа и представления в MFC — это ключевой момент при создании приложений. Она позволяет создавать

основные объекты, такие как объект приложения, классы документов и классы представлений. Объекты классов документов отвечают за внутреннее представление данных, а объекты представлений управляют отображением этих данных на экране. В свою очередь, объект приложения является инициатором и управляющим центром всего приложения.

Очень важно отметить, что поддержка контейнера OLE и сервера MFC, основанных на архитектуре документа и представления, обеспечивает возможность использования функций печати и предварительного просмотра. Это значительно упрощает процесс работы с документами и позволяет улучшить пользовательский опыт.

Более того, MFC предоставляет широкие возможности при создании объектов приложения, производных от класса CWinApp, а также основного окна приложения и других элементов. Это открывает перед разработчиками обширные перспективы для создания различных типов приложений под операционную систему Windows. В их распоряжении находится мощный инструментарий, который делает процесс разработки более гибким и эффективным.

ГЛАВА 2. РАЗРАБОТКА ПРОГРАММНОГО ПРОДУКТА

2.1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Этот уникальный программный продукт, обладающий обширным набором функциональных возможностей, предоставляет пользователю широкий спектр инструментов для управления информацией о товарном ассортименте. С его помощью можно не только регистрировать информацию о продукции, но и эффективно структурировать и организовывать накопленные данные, что является ключевым аспектом для любого бизнеса.

Важным аспектом является возможность регистрации и контроля за объемами товарных запасов на складе, что позволяет вести детальный учет наличия товаров и обеспечивает возможность оперативного реагирования на изменения в складских запасах. Кроме того, программный продукт предоставляет инструменты для реорганизации порядка представления товаров в базе данных, что способствует повышению удобства и наглядности информации для пользователя.

Интерфейс программы отличается своей удобной и функциональной конструкцией, включая в себя одно основное окно, которое является ключевым элементом для взаимодействия с данными и добавления сведений о продукции в базу данных. Это окно обеспечивает максимальную наглядность и простоту в использовании, что является важным фактором для повышения производительности работы.

В дополнение к этому, программа обладает уникальной возможностью формирования текстового файла, который может быть модифицирован вручную для корректировки возможных ошибок. Этот файл представляет собой детализированный отчет о состоянии склада, включающий в себя все необходимые сведения о товаре: его наименование, стоимость, наличие в количественном выражении и информацию о производителе. Это делает процесс учета информации более прозрачным и доступным для анализа.

2.2. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

DLL (Dynamic Link Library) – это специальная библиотека, в которую были внедрены все функциональные возможности программы [4]. Она представляет собой нечто большее, чем просто набор функций – это инструмент, способствующий повторному использованию кода различными приложениями. В контексте операционной системы Windows DLL является исполняемым файлом, содержащим общие функции и ресурсы, которые могут быть вызваны исполняемыми файлами или использованы приложениями из отдельного хранилища.

Важно отметить, что DLL не функционирует автономно, она встраивается в работу приложений, которые обращаются к ней. Преимущества использования DLL огромны: она способствует оптимизации использования памяти, экономии дискового пространства, повышению производительности приложений, снижению нагрузки на виртуальную память, обеспечивает универсальность и возможность использования в различных приложениях, а также гарантирует поддержку после продажи продукта. DLL – это не просто библиотека, это инструмент, который делает работу с программами более эффективной и удобной.

При разработке проекта первым шагом было определение общей структуры внутренних функций курсовой работы. Важным этапом является анализ файла DLL, в котором хранится базовый класс Item, содержащий ключевые характеристики продукта. На основе этого класса создается класс Tech, который представляет расширенную информацию о товаре, включая данные о производителе. Item выступает в роли фундамента для всех записей в базе данных, в то время как Tech предоставляет более детальную информацию о продукте. Оба класса включают в себя методы GetTextOut() и GetTextFile(), которые необходимы для взаимодействия с базой данных. Первый метод отвечает за вывод информации о продукте, второй же играет ключевую роль в создании текстового файла для хранения данных о содержимом базы данных.

Из вышеизложенного можно сделать вывод о том, что выполнены требования по наличию полиморфной иерархии классов в рамках данной курсовой работы. Далее стоит обратить внимание на базу данных, представленную классом DataBase. В ней содержится хранилище типа vector для всех продуктов, а также два хранилища типов map и set для удобного хранения ссылок. Таким образом, выполнено условие наличия как ассоциативных, так и обычных контейнеров, что способствует эффективной работе с данными и обеспечивает их структурированное хранение.

Одной из ключевых возможностей класса DataBase является функция ReadFile(), которая позволяет последовательно извлекать информацию из файла, разделяя данные по определенному символу-разделителю, в данном случае это запятая. Таким образом, информация поступает в хранилище. Такой подход позволяет избежать необходимости пересоздания базы данных при обновлении записей. Вместо этого реализована загрузка данных из файла.

Функция Add отвечает за добавление записей о товаре в базу данных. Стоит отметить, что Add является перегруженной функцией, что означает, что в зависимости от переданных аргументов она может выполнять различные алгоритмы. В данном контексте важно наличие или отсутствие строкового аргумента, указывающего производителя товара. По завершении работы каждой функции вызывается пара функций WriteFile() и ReadFile(), чтобы обновить текстовый файл в соответствии с внесенными изменениями. Это гарантирует стабильность работы базы данных.

Функция ShowAll предоставляет возможность формировать строковый вывод с информацией о продуктах, хранящихся в базе данных. Возвращаемые данные содержат порядковый номер товара, что облегчает отслеживание его положения в списке, а также возможность перемещения.

Символ «i» или «t» в начале строки указывает на item или tech соответственно, что помогает определить, какой объект необходимо добавить

в базу данных. За названием товара следуют цена, количество товара на складе и сведения о производителе.

При разработке было принято решение использовать MFC-приложение вместо Windows и .Net из-за его преимуществ. MFC легче в освоении, работает на C++, что устраняет необходимость настройки взаимодействия с C#. Приложение также предлагает готовые шаблоны для кнопок, текстовых окон, списков и других элементов, что значительно упрощает процесс работы. Более того, в MFC удобнее писать код для обработки событий, поэтому мой выбор остановился именно на этом приложении.

Был разработан интерфейс программы, включающий в себя окно для отображения данных, кнопки управления, поля для ввода информации о товарах и кнопку запуска процесса. Окно для вывода информации было сделано более функциональным, позволяя отображать данные на нескольких строках и использовать всю доступную площадь. Добавлена вертикальная прокрутка для удобства в случае, если введенный текст не помещается в окне. Остальные поля для ввода информации были оставлены однострочными, так как они не требуют хранения большого объема данных, в отличие от области вывода. Это улучшило функциональность программы и сделало работу с ней более удобной и эффективной.

Текстовый файл, с которым связана база данных. Именно в него производится запись данных, о новых товарах, см. (рис. 2.1.).

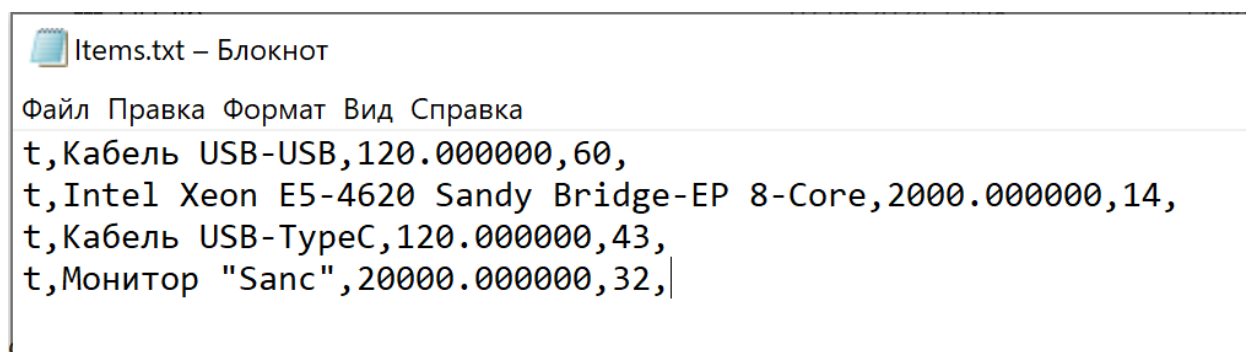


Рис. 2.1. Файл «Items.txt»

Ниже будет представлен интерфейс созданного мной приложения, с описанием особенностей работы тех или иных его элементов.

Интерфейс созданного приложения представлен на (рис. 2.2.).

The screenshot shows a Windows application window with a title bar containing the text 'Создание списка товаров'. The window has a standard Windows XP-style interface with a minimize, maximize, and close button in the top right corner. The main area of the window is divided into two sections. On the left, there is a vertical stack of buttons: 'Показать всё', 'Показать утечки', 'Кол-во товаров', 'Сортировка', 'Реверсировать', and 'Поменять местами'. Below these buttons are two empty text input fields. On the right, there is a form with five labels and corresponding input fields: 'Название', 'Цена', 'Кол-во на складе', 'Производитель (если есть, иначе оставить пустым)', and a 'Добавить' button at the bottom. To the right of the form is a large, empty rectangular area with a blue border, likely intended for displaying a list of items.

Рис. 2.2. Окно приложения

Начнем с описания события нажата кнопка «Показать всё», в таком случае срабатывает функция `void CTechShopDlg::OnBnClickedButton1()`, которая выводит список всех записей из базы данных в поле ввода. Визуализация работы данной функции представлена на (рис. 2.3.).

The screenshot shows a software window with a sidebar on the left containing the following buttons: «Показать всё» (highlighted), «Показать утечки», «Кол-во товаров», «Сортировка», «Реверсировать», «Поменять местами», and two empty input fields. The main area contains a form with the following fields: «Название», «Цена», «Кол-во на складе», and «Производитель (если есть, иначе оставить пустым)», followed by a «Добавить» button. On the right, a list of items is displayed:

- [1] Монитор "Sanc" от , 20000.000000р., на складе : 32.
- [2] Кабель USB-TypeC от , 120.000000р., на складе : 43.
- [3] Кабель USB-USB от , 120.000000р., на складе : 60.
- [4] Intel Xeon E5-4620 Sandy Bridge-EP 8-Core от , 2000.000000р., на складе : 14.

Рис. 2.3. Работа кнопки «Показать всё»

Если происходит событие нажата кнопка «Показать утечки», тогда срабатывает функция `void CTechShopDlg::OnBnClickedButton2()`, проверяющая программу на наличие утечек памяти и выводящая результат в поле ввода, что представлено на (рис. 2.4.).

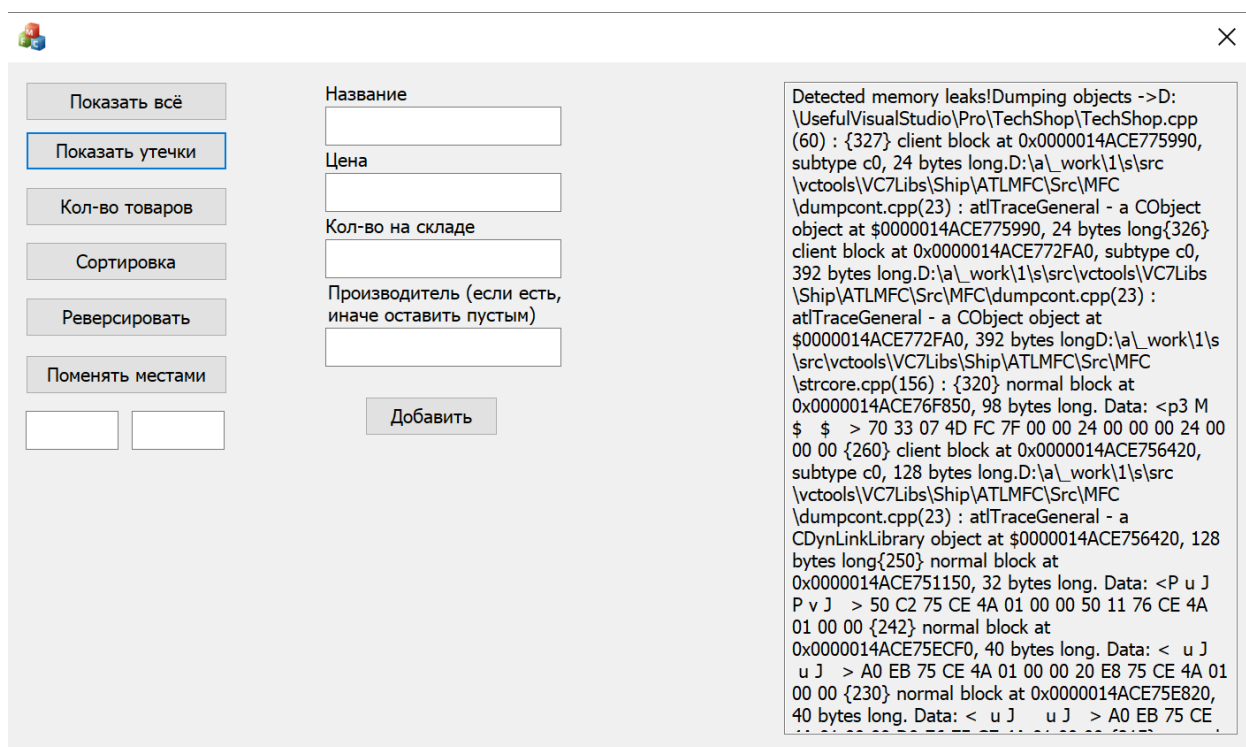


Рис. 2.4. Работа кнопки «Показать утечки»

Если происходит событие нажата кнопка «Количество товаров», то срабатывает функция `void CTechShopDlg::OnBnClickedButton3()`, подсчитывающая количество товаров в базе данных и выводящая результат в поле ввода, что представлено на (рис. 2.5.).

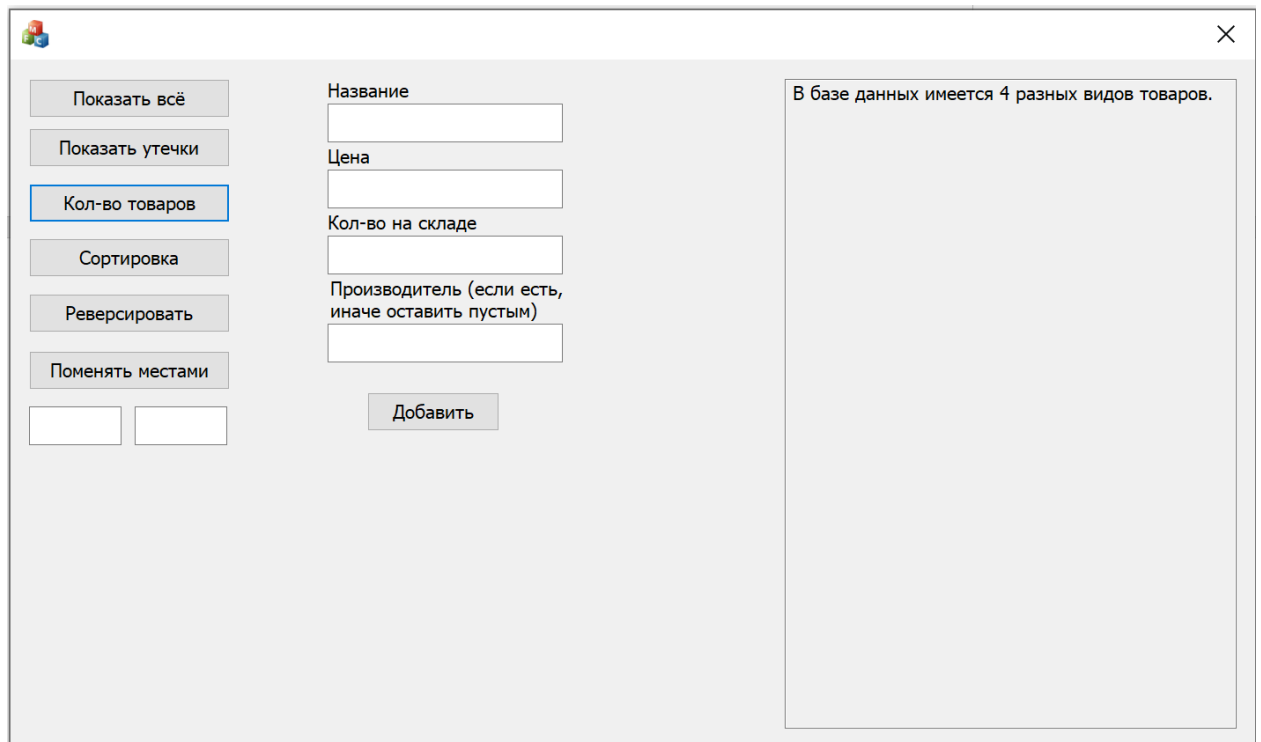
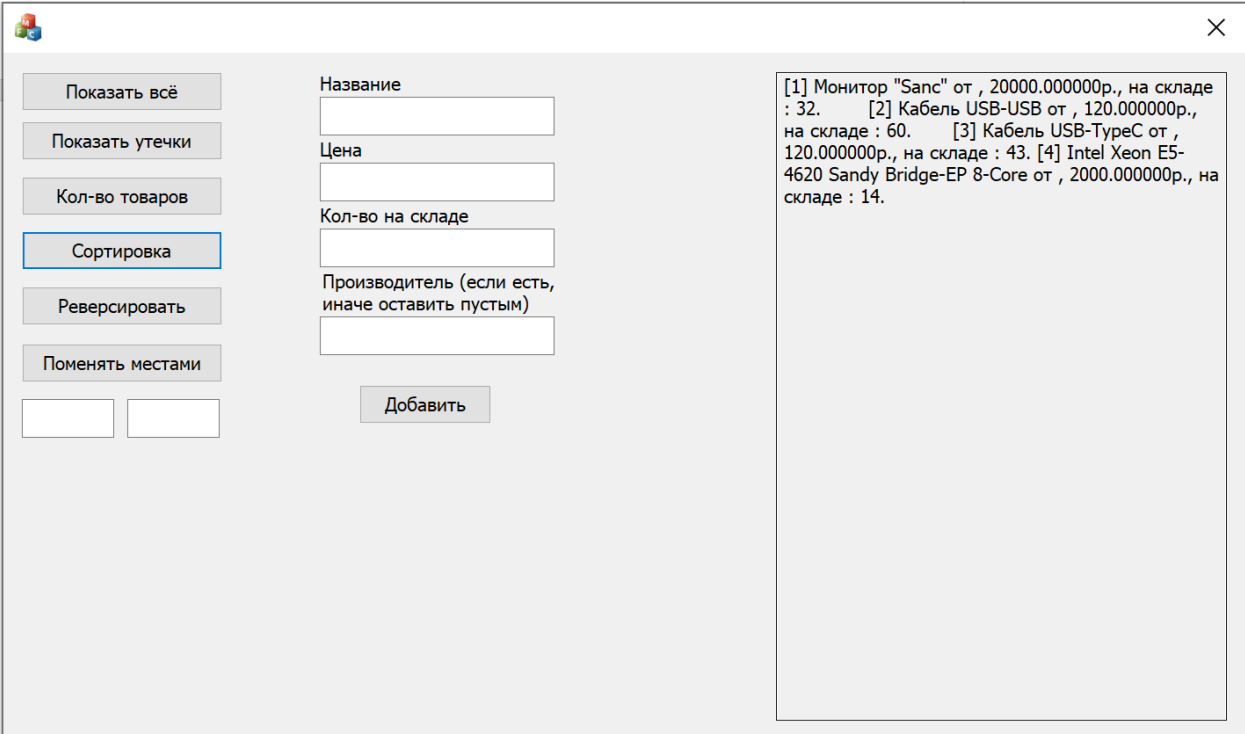


Рис. 2.5. Работа кнопки «Количество товаров»

Если происходит событие нажата кнопка «Сортировка», то срабатывает функция `void CTechShopDlg::OnBnClickedButton4()`, выводящая отсортированный список, состоящий из всех товаров базы данных, в поле ввода, что представлено на (рис. 2.6.).



Показать всё

Показать утечки

Кол-во товаров

Сортировка

Реверсировать

Поменять местами

Название

Цена

Кол-во на складе

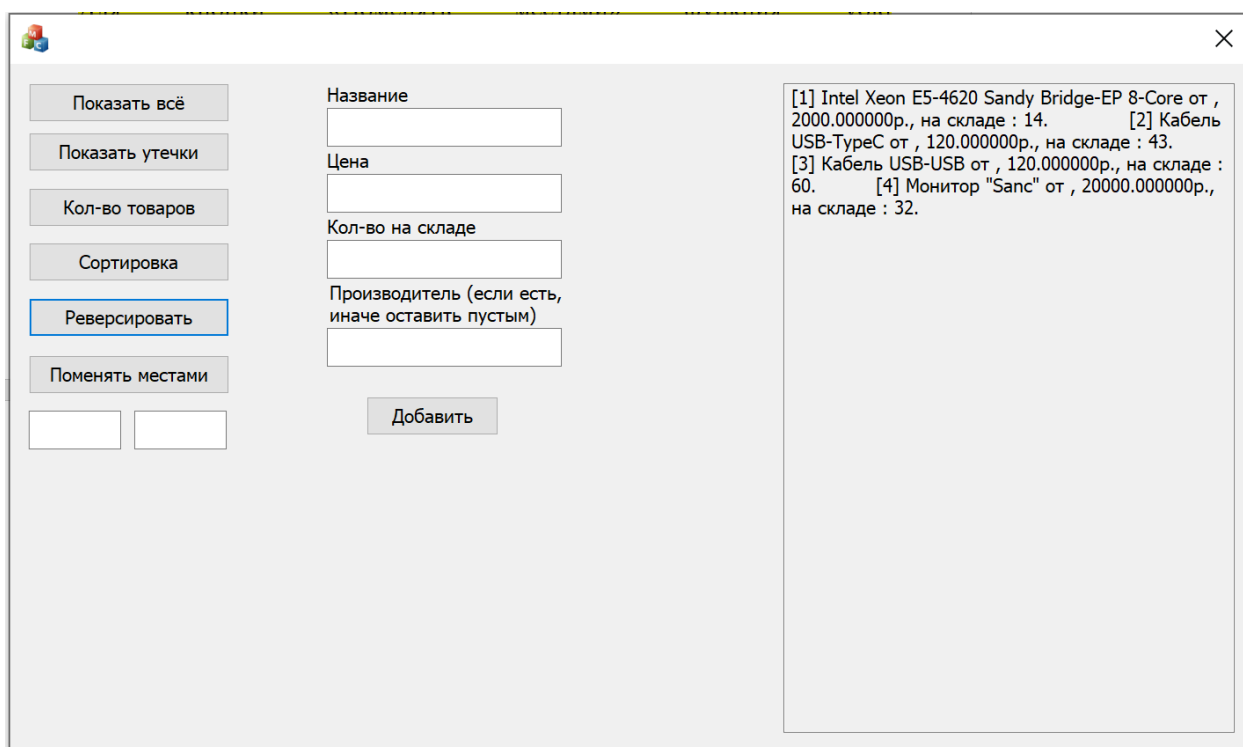
Производитель (если есть, иначе оставить пустым)

Добавить

[1] Монитор "Sanc" от , 20000.000000р., на складе : 32. [2] Кабель USB-USB от , 120.000000р., на складе : 60. [3] Кабель USB-TypeC от , 120.000000р., на складе : 43. [4] Intel Xeon E5-4620 Sandy Bridge-EP 8-Core от , 2000.000000р., на складе : 14.

Рис. 2.6. Работа кнопки «Сортировка»

Если происходит событие нажата кнопка «Реверсировать», то срабатывает функция `void CTechShopDlg::OnBnClickedButton5()`, выводящая отсортированный в обратном порядке (реверсивном) список, состоящий из всех товаров базы данных, в поле ввода, что представлено на (рис. 2.7.).



The screenshot shows a software window with a title bar and a close button. Inside, there is a form with several buttons and input fields. The buttons are: "Показать всё", "Показать утечки", "Кол-во товаров", "Сортировка", "Реверсировать" (highlighted with a blue border), "Поменять местами", and "Добавить". There are also two empty input fields below the "Поменять местами" button. The form has three main sections: "Название", "Цена", and "Кол-во на складе", each with an input field. Below these is a section for "Производитель (если есть, иначе оставить пустым)" with an input field. On the right side, there is a text area containing a list of items:

- [1] Intel Xeon E5-4620 Sandy Bridge-EP 8-Core от , 2000.000000р., на складе : 14.
- [2] Кабель USB-TypeC от , 120.000000р., на складе : 43.
- [3] Кабель USB-USB от , 120.000000р., на складе : 60.
- [4] Монитор "Sanc" от , 20000.000000р., на складе : 32.

Рис. 2.7. Работа кнопки «Реверсировать»

Если происходит событие нажата кнопка «Поменять местами», то срабатывает функция `void CTechShopDlg::OnBnClickedButton6()`, меняющая местами две записи о товарах в базе данных, определяющихся по порядковым номерам, введенным в поля ввода под кнопкой, что представлено на (рис. 2.8.).

Показать всё

Показать утечки

Кол-во товаров

Сортировка

Реверсировать

Поменять местами

1 4

Название

Цена

Кол-во на складе

Производитель (если есть, иначе оставить пустым)

Добавить

[1] Монитор "Sanc" от , 20000.000000р., на складе : 32. [2] Кабель USB-TypeC от , 120.000000р., на складе : 43. [3] Кабель USB-USB от , 120.000000р., на складе : 60. [4] Intel Xeon E5-4620 Sandy Bridge-EP 8-Core от , 2000.000000р., на складе : 14.

Рис. 2.8. Работа кнопки «Поменять местами»

Если происходит событие нажата кнопка «Добавить», то срабатывает функция `void CTechShopDlg::OnBnClickedButton8()`, добавляющая запись о товаре, опираясь на введенные данные в соответствующие поля, что продемонстрировано на (рис. 2.9.).

The screenshot shows a software window with a sidebar on the left containing several buttons: "Показать всё", "Показать утечки", "Кол-во товаров", "Сортировка", "Реверсировать", "Поменять местами", and two empty input fields. The main area contains a form with the following fields: "Название" (filled with "Колонки 'Зевс'"), "Цена" (filled with "35000"), "Кол-во на складе" (filled with "40"), and "Производитель (если есть, иначе оставить пустым)". Below these fields is a "Добавить" button. To the right of the form, a message box states: "В базу данных добавлен Колонки 'Зевс'".

Рис. 2.9. Работа кнопки «Добавить»

ЗАКЛЮЧЕНИЕ

В рамках проведённой работы был разработан уникальный и специализированный инструмент, который предназначен для эффективного управления информационными ресурсами, хранящимися в базе данных электронного магазина. В процессе создания данной программы был применён комплексный подход, основанный на фундаментальных принципах объектно-ориентированного программирования. Этот метод разработки позволил создать программное обеспечение, обладающее высокой степенью модульности и способное к легкому расширению функциональности без необходимости внесения значительных изменений в уже написанный код. Кроме того, была использована технология C++/CLI, которая интегрирована в обширный и многофункциональный .NET фреймворк.

В дополнение к вышеуказанному, интеграция с динамически подключаемыми библиотеками (DLL) открыла перед разработчиками широкие возможности для более индивидуального и творческого подхода к настройкам пользовательского интерфейса. Это дало возможность создать интерфейс, который будет максимально удобен и интуитивен для пользователей, а также обеспечит гибкость и адаптивность к различным требованиям и предпочтениям.

В процессе реализации данного проекта были не только освоены, но и глубоко изучены ключевые навыки, связанные с проектированием и разработкой программных продуктов с применением объектно-ориентированного подхода. Разработчики также получили ценный опыт в работе с динамически подключаемыми библиотеками, что значительно расширило их профессиональные возможности. В результате этой работы был создан инновационный и интуитивно понятный пользовательский интерфейс, который был реализован на платформе .NET, что обеспечило высокую степень интеграции и совместимости с уже существующими системными компонентами.

СПИСОК ЛИТЕРАТУРЫ

1. PHP.ZONE [Электронный ресурс] Режим доступа:
<https://php.zone/post/dlya-chego-pridumali-oop>
2. Metanit.com [Электронный ресурс] Режим доступа:
<https://metanit.com/cpp/tutorial/1.1.php>
3. Tproger [Электронный ресурс] Режим доступа:
<https://tproger.ru/tag/oop>
4. Микрософт [Электронный ресурс] Режим доступа:
<https://learn.microsoft.com/ru-ru/visualstudio/get-started/visual-studio-ide?view=vs-2019#build-your-app>
5. Яндекс Практикум [Электронный ресурс] Режим доступа:
<https://practicum.yandex.ru/blog/obektno-orientirovannoe-programmirovanie/#populyarnye-yazyki-oop>

ПРОГРАММНЫЙ КОД

```

#pragma once
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <set>
#include <algorithm>
#include <map>
#include <iterator>

using namespace std;

class Item {
public:
    string Name;
    double Price;
    int Amount;
    Item(string n, double p, int a) {
        Name = n;
        Price = p;
        Amount = a;
    }
    Item() {};
    virtual ~Item() {};
    virtual string GetTextOut() {
        return Name + ", " + to_string(Price) + "p., на
складе: " + to_string(Amount) + ".";
    }
    virtual string GetTextFile() {
        return "i," + Name + "," + to_string(Price) + "," +
to_string(Amount) + ",";
    }
};

class Tech : public Item {
public:
    string Producer;
    Tech(string n, double p, int a, string pr) {
        Name = n;
        Price = p;
        Amount = a;
        Producer = pr;
    }
    ~Tech() {};
    virtual string GetTextOut() {

```

```

        return Name + " от " + Producer + ", " +
to_string(Price) + "p., на складе : " + to_string(Amount) + ".";
    }
    virtual string GetTextFile() {
        return "t," + Name + "," + to_string(Price) + "," +
to_string(Amount) + ",";
    }
};

class DataBase {
public:
    vector<Item*> All;
    map<string, set<Item*>> TypesAndItems;
    set<string> Types;
    void ReadFile() {
        Types.clear();
        Types.insert("Items");
        Types.insert("Tech");
        for (set<string>::iterator it = Types.begin(); it !=
Types.end(); it++) {
            TypesAndItems[*it].clear();
        }
        for (int i = 0; i < All.size(); i++) {
            delete All[i];
        }
        All.clear();
        ifstream file("Items.txt");
        string line;
        if (file) {
            while (!file.eof()) {
                getline(file, line);
                string Title = "", Producer = "",

buf = "";

                int amount;
                double price;
                int i = 2;
                for (; i < line.size() - 1; i++) {
                    Title += line[i];
                    if (line[i + 1] == ',') {
                        i += 2;
                        break;
                    }
                }
                for (; i < line.size() - 1; i++) {
                    buf += line[i];
                    if (line[i + 1] == ',') {
                        price =

stod(buf.c_str());

                        buf = "";
                        i += 2;
                        break;
                    }
                }
            }
        }
    }
};

```

```

        }
    }
    for (; i < line.size() - 1; i++) {
        buf += line[i];
        if (line[i + 1] == ',') {
            amount =
stoi(buf.c_str());

            buf = "";
            i += 2;
            break;
        }
    }
    if (line[0] == 'i') {
        TypesAndItems["Items"].insert(new Item(Title, price, amount));
    }
    if (line[0] == 't') {
        for (; i < line.size() - 1;
i++) {
            Producer +=
line[i];
            if (line[i + 1] ==
',') {
                break;
            }
        }
        TypesAndItems["Tech"].insert(new Tech(Title, price, amount,
Producer));
    }
}
file.close();
set_union(TypesAndItems["Items"].begin(),
TypesAndItems["Items"].end(), TypesAndItems["Tech"].begin(),
TypesAndItems["Tech"].end(), back_inserter(All));
}
void WriteFile() {
    ofstream out("Items.txt");
    for (vector<Item*>::iterator it = All.begin(); it !=
All.end(); it++) {
        if (it == All.begin()) {
            out << (*it)->GetTextFile();
        }
        else {
            out << "\n" << (*it)->GetTextFile();
        }
    }
    out.close();
}
DataBase() {

```

```

        Types.insert("Items");
        Types.insert("Tech");
        ReadFile();
    }
    ~DataBase() {
        WriteFile();
        for (set<string>::iterator it = Types.begin(); it !=
Types.end(); it++) {
            TypesAndItems[*it].clear();
        }
        for (int i = 0; i < All.size(); i++) {
            delete All[i];
        }
        All.clear();
    }
    string ShowAll() {
        string str = "";
        int i = 0;
        for (vector<Item*>::iterator it = All.begin(); it !=
All.end(); it++) {
            i++;
            if (it == All.begin()) {
                str += "[" + to_string(i) + "] ";
                str += (*it)->GetTextOut();
            }
            else {
                str += '\t';
                str += "[" + to_string(i) + "] ";
                str += (*it)->GetTextOut();
            }
        }
        return str;
    }

    void Sort() {
        sort(All.begin(), All.end(), [](Item* a, Item* b) {
            return a->Name > b->Name;
        });
    }
    void Reverse() {
        reverse(All.begin(), All.end());
    }
    void Swap(int a, int b) {
        iter_swap(All.begin() + a - 1, All.begin() + a - 1);
    }
    void Unique() {
        unique(All.begin(), All.end());
    }
    void Add(string Title, double price, int amount) {
        Item* n = new Item(Title, price, amount);
    }

```

```

        pair<vector<Item*>::iterator,
vector<Item*>::iterator> p = equal_range(All.begin(), All.end(), n);
        All.insert(p.first, n);
        WriteFile();
        ReadFile();
    }
    void Add(string Title, double price, int amount, string
producer) {
        Tech* n = new Tech(Title, price, amount, producer);
        pair<vector<Item*>::iterator,
vector<Item*>::iterator> p = equal_range(All.begin(), All.end(), n);
        All.insert(p.first, n);
        WriteFile();
        ReadFile();
    }
};

```

ПРОГРАММНЫЙ КОД

```

#include "Dll.h"
#include "wtypes.h"
#include "Tech.h"

DataBase DB;

void UpdateFile() {
    DB.WriteFile();
    DB.ReadFile();
};

string ShowAll() {
    return DB.ShowAll();
};

std::string CheckMemory() {
    DB.WriteFile();
    for (int i = 0; i < DB.All.size(); i++) {
        delete DB.All[i];
    }
    DB.All.clear();
    DB.Types.clear();
    HANDLE hCrtLog = CreateFile(TEXT("crt.log"), GENERIC_WRITE,
FILE_SHARE_WRITE, NULL, CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
    _CrtSetReportMode(_CRT_WARN, _CRTDBG_MODE_FILE);
    _CrtSetReportFile(_CRT_WARN, _CRTDBG_FILE_STDOUT);
    if (hCrtLog != INVALID_HANDLE_VALUE)
    {
        _CrtSetReportMode(_CRT_WARN, _CRTDBG_MODE_FILE);
        _CrtSetReportMode(_CRT_ERROR, _CRTDBG_MODE_FILE);
        _CrtSetReportFile(_CRT_WARN, hCrtLog);
        _CrtSetReportFile(_CRT_ERROR, hCrtLog);
    }
    _CrtDumpMemoryLeaks();
    if (hCrtLog != INVALID_HANDLE_VALUE)
        CloseHandle(hCrtLog);
    std::ifstream file("crt.log");
    std::string g = "";
    if (file.is_open()) {
        while (!file.eof()) {
            std::string line;
            getline(file, line);
            g += line;
        }
    }
    DB.ReadFile();
    return g;
}

```

```

}

string GetCount() {
    int amount = std::count_if(DB.All.begin(), DB.All.end(), [](Item*
a) {
        if (a->GetTextFile() != "") return 1;
        else return 0;
    });
    return std::to_string(amount);
}

bool Reverse() {
    DB.Reverse();
    return true;
}

bool Sort() {
    DB.Sort();
    return 1;
}

bool Swap(int a, int b) {
    a = a - 1;
    b = b - 1;
    if (a >= 0 and b >= 0 and a < DB.All.size() and b < DB.All.size())
    {
        iter_swap(DB.All.begin() + a, DB.All.begin() + b);
    }
    return true;
};

void Add(string Title, double price, int amount) {
    DB.Add(Title, price, amount);
}

void Add(string Title, double price, int amount, string producer) {
    DB.Add(Title, price, amount, producer);
}

void Delete(int a) {
    a = a - 1;
    if (a >= 0 and a < DB.All.size()) {
        delete DB.All[a];
    }
    DB.All.erase(DB.All.begin() + a);
};

```

ПРОГРАММНЫЙ КОД

```
#pragma once
#include <string>

using namespace std;

#ifdef DLL_EXPORTS
#define DLL_API __declspec(dllexport)
#else
#define DLL_API __declspec(dllimport)
#endif

DLL_API void UpdateFile();
DLL_API string ShowAll();
DLL_API string CheckMemory();
DLL_API string GetCount();
DLL_API bool Reverse();
DLL_API bool Sort();
DLL_API bool Swap(int a, int b);
DLL_API void Add(string Title, double price, int amount);
DLL_API void Add(string Title, double price, int amount, string
producer);
DLL_API void Delete(int a);
```